



AMERICAN INTERNATIONAL UNIVERSITY-BANGLADESH (AIUB)

Faculty of Science and Technology (FST)
Department of Computer Science (CS)
Undergraduate Program

COURSE PLAN	SEMESTER: Summer 2025-2026
I. Course Core and Title CSC 4273 Software Architecture & Design Patterns II. Credit 3 credit hours (3 hours of theory per week) III. Nature Core Course for Software Engineering major IV. Prerequisite CSC 3215: Web Technologies	V. Vision: Our vision is to be the preeminent Department of Computer Science through creating recognized professionals who will provide innovative solutions by leveraging contemporary research methods and development techniques of computing that is in line with the national and global context. VI. Mission: The mission of the Department of Computer Science of AIUB is to educate students in a student-centric dynamic learning environment; to provide advanced facilities for conducting innovative research and development to meet the challenges of the modern era of computing, and to motivate them towards a life-long learning process.

VII - Course Description

This course provides a comprehensive theoretical and applied understanding of software architecture and design patterns. Students learn to design scalable, maintainable, and robust software systems by studying classical architectural styles alongside cutting-edge architectures for AI-native applications, decentralized systems, and large-scale distributed platforms. Core design patterns (creational, structural, behavioral) are studied alongside modern distributed patterns such as CQRS, Saga, Circuit Breaker, and AI agent orchestration architectures. Emphasis is placed on non-functional quality attributes for large-scale systems, architectural decision documentation, trade-off analysis, and critical evaluation of real-world systems.

Students will be able to:

- Understand what software architecture is and how it differs from design, and apply architectural thinking to real engineering problems.
- Compare and select appropriate architectural styles: layered, MVC, microservices, event-driven, serverless, and decentralized.
- Apply classic and modern design patterns to solve recurring software engineering problems.
- Analyze non-functional attributes (scalability, availability, security, performance, resilience) for large-scale distributed systems.
- Design architectures for AI agent systems including multi-agent orchestration, RAG pipelines, and tool-use patterns.
- Evaluate decentralized application (dApp) architectures including smart contract patterns, on-chain/off-chain decisions, and Web3 integrations.
- Document architectural decisions using ADRs, C4 models, and UML diagrams.

Develop and present a semester-long group architecture project integrating multiple design patterns and documented decisions.

VIII - Course outcomes (CO) Matrix

By the end of this course, students should be able to:

COs*	CO Description	Level of Domain***			PO Assessed****
		C	P	A	
CO1	Analyze quality attributes (scalability, availability, security, resilience, maintainability) and trade-offs in software architecture decisions considering societal and environmental contexts, including large-scale distributed systems.	4			PO-c-1
CO2	Classify appropriate architectural styles (layered, MVC, microservices, event-driven, AI-agent, decentralized) and design patterns for given problem contexts, including AI-native and Web3 applications.	4			PO-c-1
CO3**	Illustrate the structure and behavior of a multi-tier software system using UML diagrams (component, deployment, sequence), C4 models, and Architecture Decision Records (ADRs).	4			PO-c-2
CO4**	Develop a multi-tier or microservice-based software architecture as a group member/leader, integrating at least three design patterns, documented ADRs, and addressing at least two non-functional quality attributes.		4		PO-k-2

C: Cognitive; P: Psychomotor; A: Affective Domain

* CO assessment method and rubric of COs assessment is provided in later section

** COs will be mapped with the Program Outcomes (POs) for PO attainment

*** The numbers under the 'Level of Domain' columns represent the level of Bloom's Taxonomy each CO corresponds to.

**** The numbers under 'PO Assessed' column represent the POs each CO corresponds to.

IX - Topics to be covered in the class: *

Time Frame	CO Mapped	Topics	Teaching Activities	Assessment Strategy(s)
Week 1	CO1, CO2	Course Introduction & OBE Discussion; Mission & Vision of AIUB; What is Software Architecture and how does it differ from Design? SOLID principles recap; Introduction to Quality Attributes: scalability, availability, security, maintainability	Lecture, discussion	Homework: Reflect on the architecture of a system you use daily
Week 2	CO1, CO2	Foundational Architectural Styles: Layered (N-Tier), MVC, Client-Server; Monolith vs. Service-Oriented; UML Component & Deployment Diagrams; C4 model overview; Case study: Amazon/Facebook early monolith	Lecture, diagram workshop	In-class exercise: Draw a C4 context diagram for a given system
Week 3	CO2, CO3	Creational Design Patterns: Singleton, Factory Method, Abstract Factory, Builder; Trade-offs and when to use each; Refactoring toward creational patterns	Lecture, pattern comparison	Assignment 1: Written analysis of pattern applicability in a given scenario
Week 4	CO2, CO3	Structural Design Patterns: Adapter, Facade, Proxy, Decorator, Composite; Identifying patterns in real systems; Trade-offs between structural choices	Lecture, pair analysis	Quiz 1 (Weeks 1–3)
Week 5	CO2, CO3	Behavioral Design Patterns: Strategy, Observer, Command, Template Method, State; Event-driven patterns; Team activity: design a notification system using Observer	Lecture, pattern roleplay	Assignment 2: Compare two behavioral patterns applied to a real-world design scenario
Week 6	CO1, CO2, CO3	Distributed & Cloud-Native Architectures: Microservices principles; Event-Driven Architecture (EDA); API Gateway; Circuit Breaker and Saga patterns; Trade-offs vs. monoliths; Case study: Netflix/Uber migration	Lecture, case study discussion	Quiz 2 (Weeks 4–5); Case Study Draft (individual): analyze a real-world architectural decision

Midterm (Week 7)				
Individual viva/demo sessions with faculty+ Case Study Report submission				
Week 8	CO1, CO3	Non-Functional Attributes (NFAs) for Large-Scale Systems: Scalability (horizontal vs. vertical); Availability & Reliability (CAP theorem, replication basics); Fault Tolerance and Resilience patterns; Load balancing overview; Introduction to ATAM (Architecture Trade-off Analysis Method)	Lecture, workshop	Homework: Apply ATAM to a given system scenario
Week 9	CO1, CO3	NFAs Continued – Performance, Security & Documentation: Performance and capacity planning basics; Security by design (Zero Trust, API security patterns); Observability overview (logging, metrics, tracing); Architecture Decision Records (ADRs) and C4 model in depth	Lecture, workshop	Project ADR Draft due (group); Quiz 3 (Weeks 6, 8)
Week 10	CO1, CO2, CO4	Modern Application Paradigms – Part 1: Centralized vs. Decentralized Architectures: • Characteristics of centralized systems; why decentralization? • Blockchain fundamentals: how it works, consensus mechanisms, how it differs from traditional databases and architectures • Architectural comparison: traditional web app vs. blockchain-backed dApp; on-chain vs. off-chain decisions IoT Application Architecture: • Device–Edge–Cloud layered model • Publish-subscribe and data pipeline patterns for IoT • Architectural trade-offs in resource-constrained environments	Lecture, diagram comparison, case study discussion	Assignment 3: Architectural comparison – choose a domain and compare centralized vs. decentralized design (diagram + short justification)
Week 11	CO2, CO4	Modern Application Paradigms – Part 2: Agentic Applications & AI Pipelines: • What is an AI agent? Perceive–Reason–Act loop • Agentic pipeline architecture: planner, tools, memory, executor • RAG (Retrieval-Augmented Generation) pipeline design • Multi-agent orchestration: overview of patterns • Comparing traditional request-response apps with agentic systems • Key architectural considerations: memory management, observability, safety guardrails	Lecture, demo, group discussion	Homework: Sketch an agentic application architecture diagram for a given use case
Week 12	CO4	Project Work & Architecture Documentation Session: In-class group consultations and architecture reviews; Peer review of ADRs and diagrams; Writing a research proposal in software architecture (problem statement, literature review, proposed solution); Finalizing architectural decisions for the project	Mentoring, peer review, consultations	Research Proposal draft due (group); Project Checkpoint (architecture description + diagrams) submitted
Week 13	CO3, CO4	Architecture Review & Consolidation: Serverless and FaaS overview; Zero-downtime deployment strategies (Blue-Green, Canary); Recap of architectural styles, patterns, and NFAs covered; Open discussion: student-chosen emerging architecture topic; Project finalization guidance	Lecture, student-led discussion	Quiz 4 (Weeks 10–12)
Final term (Week 14)				
Final Presentation/Demo + Project Report + Research Proposal				

* The faculty reserves the right to change, amend, add, or delete any of the contents.

X - Mapping of PO to Courses and K, P, A

PO Indicator ID	PO Indicators Definition (As per the requirement of WKs)	Domain	K	P	A
PO-c-1	Design solutions for component of a complex engineering problem considering public health and safety	Cognitive Level 4 (Analyzing)	K5		
PO-c-2	Develop process for complex engineering problems considering cultural and societal factors.	Cognitive Level 4 (Analyzing)	K5	P1 P3 P7	
PO-k-3	Manage multi-disciplinary components of a computer science and engineering project as a member/leader	Psychomotor Level 4 (Articulation)			

XI – K, P, A Definitions

Indicator	Title	Description
K5	Engineering Design	Knowledge that supports engineering design in a practice area
P1	Depth of knowledge required	Cannot be resolved without in-depth engineering knowledge at the level of one or more of K3, K4, K5, K6 or K8 which allows a fundamentals-based, first principles analytical approach
P3	Depth of analysis required	Have no obvious solution and require abstract thinking, originality in analysis to formulate suitable models
P7	Interdependence	Are high level problems including many component parts or sub-problems

XII – Mapping of CO Assessment Method and Rubric

The mapping between Course Outcome(s) (COs) and The Selected Assessment method(s) and the mapping between Assessment method(s) and Evaluation Rubric(s) is shown below:

COs	Description	Mapped POs	Assessment Method	Assessment Rubric
CO1	Analyze quality attributes and trade-offs	PO-c-1	Case Study Report + Research Proposal (literature review section)	Rubric for CO1 (Section XIII)
CO2	Classify architectural styles and design patterns	PO-c-1	Quizzes + Assignments (pattern identification exercises)	Rubric for CO2 (Section XIII)
CO3	Illustrate multi-tier system using UML and ADRs	PO-c-2	Project Report (diagrams + ADRs) + Assignment 2 & 3	Rubric for CO3 (Section XIII)
CO4	Develop multi-tier application as group member/leader	PO-k-3	Final Project (architecture docs + ADRs + GitHub + presentation)	Rubric for CO4 (Section XIII)

XIII – Evaluation and Assessment Criteria**CO1 [PO-c1]: Analyze quality attributes and trade-offs in architectural decisions**

Assessment Attribute/Criteria	Missing/Incorrect (0)	Inadequate (1)	Satisfactory (2)	Excellent (3)
Case Study Report & Viva (e.g. analyze Netflix's or Uber's architecture)	No submission or entirely off-topic	Superficial; lists attributes without justification	Clearly identifies trade-offs (e.g., scalability vs. consistency) with reasoned arguments	Exceptional; references ATAM; quantifies trade-offs; connects to societal impact
Research Proposal – Problem Statement & Literature Review	Missing entirely	Vague problem statement; few relevant references	Clear problem statement; good coverage of existing architectural solutions	Insightful; identifies gaps; proposes novel architectural idea with strong justification

CO2 [PO-c1]: Classify architectural styles and design patterns

Assessment Attribute/Criteria	Missing/Incorrect (0)	Inadequate (1)	Satisfactory (2)	Excellent (3)
Quiz (MCQ + short answer)	< 40% correct	40–60% correct	60–80% correct	> 80% correct
Assignment (pattern classification)	Not submitted or completely wrong	Names patterns but misapplies them to the scenario	Correctly classifies and selects appropriate pattern/style	Justifies selection with strengths/weaknesses; compares alternatives

CO3 [PO-c2]: Illustrate multi-tier system using UML and ADRs

Assessment Attribute/Criteria	Missing/Incorrect (0)	Inadequate (1)	Satisfactory (2)	Excellent (3)
UML Diagrams (component, sequence, deployment)	No diagrams or incorrect notation	Diagrams incomplete or internally inconsistent	Correctly drawn; show multi-tier structure and key interactions	Professional quality; consistent notation; all necessary views represented
Architecture Decision Records (ADRs)	No ADRs submitted	ADRs missing rationale or alternatives	ADRs include context, decision, consequences, and alternatives	Excellent documentation with trade-off analysis and full traceability

CO4 [PO-k2]: Develop multi-tier software architecture as group member/leader

Assessment Attribute/Criteria	Missing/Incorrect (0)	Inadequate (1)	Satisfactory (2)	Excellent (3)
-------------------------------	-----------------------	----------------	------------------	---------------

Collaborative Teamwork (GitHub / shared docs)	No contribution; no version control	Minimal commits; passive participation	Regular contributions; participates in reviews	Active leader; coordinates team; integrates and resolves conflicts
Final Presentation/Demo (Week 14 viva)	Missing or off-topic	Poor delivery; cannot answer questions	Clear; covers architecture, patterns, decisions; answers questions confidently	Engaging; deep technical insight; excellent Q&A; demonstrates strong understanding
Final Project Report	No report or irrelevant content	Missing key sections (no ADRs, no pattern justification)	Complete: context, architectural style, pattern descriptions, UML, ADRs, references	Exceptional: includes trade-off analysis, alternatives considered, professionally structured

XIV- Course Requirements

- Students must attend at least 80% of classes.
- Active participation in lectures, case study discussions, and group work is required.
- Four quizzes (pattern identification and architecture knowledge) spread across both terms.
- Three individual assignments: creational pattern analysis, behavioral pattern comparison, and architectural comparison (centralized vs. decentralized).
- One individual Case Study Report + Viva/Demo (Week 7, mid-term exam week): deep analysis of a real-world architecture, defended in a faculty viva session.
- One group Research Proposal on a chosen architecture topic (AI agents, decentralized systems, large-scale NFAs, or IoT).
- Semester-long group project (3–4 members) with the following milestones:
 - ADR Draft (Week 9)
 - Project Checkpoint – architecture description + initial diagrams (Week 12)
 - Final Presentation/Demo (Week 14, final exam week) – 15 minutes per group
 - Final Project Report (Week 14, final exam week)
- Submission of all deliverables must be on time; late submissions are subject to penalty as per course policy.

XV – Evaluation & Grading System*

The following grading system will be strictly followed in this class.

MID TERM		FINAL TERM	
Attendance	5%	Attendance	5%
Assignment	10%	Assignment	10%
Quiz	10%	Quiz	10%
Case Study	15%	Research Proposal	10%
		Project Report & Presentation	25%
Total	40%	Total	60%
Grand Total 100% = 40% of Midterm + 60% of Final Term			

Letter	Grade Point	Numerical %
A+	4.00	90-100
A	3.75	85 - < 90
B+	3.50	80 - < 85
B	3.25	75 - < 80

C+	3.00	70 - < 75
C	2.75	65 - < 70
D+	2.50	60 - < 65
D	2.25	50 - < 60
F	0.00	< 50
I		Incomplete
W		Withdrawal
UW		Unofficially Withdrawal

* The evaluation system will be strictly followed as par with the AIUB grading policy.

* CO attainment will be achieved with 60% of the evaluation marks.

XVI – Textbook/ References

Core Textbooks

- Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (1994). Design Patterns: Elements of Reusable Object-Oriented Software. Addison-Wesley. (GoF – Gang of Four)
- Bass, L., Clements, P., & Kazman, R. (2012). Software Architecture in Practice (3rd ed.). Addison-Wesley.
- Martin, R. C. (2017). Clean Architecture: A Craftsman's Guide to Software Structure and Design. Prentice Hall.
- Freeman, E., & Robson, E. (2020). Head First Design Patterns (2nd ed.). O'Reilly.
- Richards, M., & Ford, N. (2020). Fundamentals of Software Architecture. O'Reilly.
- Newman, S. (2021). Building Microservices (2nd ed.). O'Reilly.

Supplementary – AI Agent & Large-Scale Systems

- Lim, T., et al. (2024). LLM-Powered Agents: Architectural Patterns and Best Practices. (Microsoft Research / arXiv)
- Wieringa, R. (2014). Design Science Methodology for Information Systems and Software Engineering. Springer.
- Kleppmann, M. (2017). Designing Data-Intensive Applications. O'Reilly. (Event sourcing, CQRS, distributed systems)
- Google Cloud Architecture Framework; Microsoft Azure Architecture Center; AWS Well-Architected Framework – all available online.

Supplementary – Decentralized Systems

- Antonopoulos, A. M., & Wood, G. (2018). Mastering Ethereum. O'Reilly.
- Openness, L. et al. (2022). Smart Contract Design Patterns – Solidity Documentation. ethereum.org.
- IPFS Documentation: <https://docs.ipfs.tech>

Online Resources

- Refactoring.Guru – Design Patterns: <https://refactoring.guru/design-patterns>
- Martin Fowler's Architecture Blog: <https://martinfowler.com>
- Microsoft Architecture Guides: <https://learn.microsoft.com/azure/architecture>
- OpenTelemetry Documentation: <https://opentelemetry.io/docs>
- LangChain / LangGraph Documentation: <https://docs.langchain.com>

XVII - List of Faculties Teaching the Course

FACULTY NAME	SIGNATURE
Mr. Md. Al-Amin	

XVIII – Verification

Prepared by: ----- Md. Al-Amin <i>Course Convener</i> Date: 03/06/2026	Moderated by: ----- Dr. M. Mahmudul Hasan <i>Point Of Contact</i> <i>OBE Implementation Committee</i> Date:.....	Checked by: ----- Dr. Muhammad Firoz Mridha <i>Head (Undergraduate Program)</i> <i>Department of Computer Science</i> Date:.....
Verified by: ----- Dr. Md. Saef Ullah Miah <i>Director</i> <i>Faculty of Science & Information Technology</i> Date:.....	Certified by: ----- Dr. Khandaker Tabin Hasan <i>Associate Dean,</i> <i>Faculty of Science & Information Technology</i> Date:.....	Approved by: ----- Prof. Dr. Dip Nandi <i>Dean,</i> <i>Faculty of Science & Information Technology</i> Date:.....